

Unit-4: Greedy Algorithms

Optimization problems and optimal solutions

- An optimization problem is the problem of finding the best solution from all feasible solutions.
- Optimization : maximization or minimization
- Optimal solution is a feasible solution where the objective function reaches its minimum or maximum value.
- Example: maximizing profit, minimizing loss, minimizing power consumption, minimizing travel cost, maximizing rewards.

Introduction to greedy algorithms

- Greedy paradigm is used to develop the algorithm to obtain the solution for some optimization problems (For example, computing shortest path in graph, minimum spanning tree)
- Algorithms for optimization problems typically go through a sequence of steps with a set of choices at each step.
- A greedy algorithm, as the name suggests, always makes the choice that seems to be the best at that moment. This means that it makes a locally-optimal choice in the hope that this choice will lead to a globally-optimal solution.
- While applying greedy approach some objective function is identified that is helpful to capture the optimality. To obtain the solution, candidate items are evaluated with objective function and the item that best optimizes the objective function is taken as next candidate to be added to the partial solution of the problem.
- As being greedy, the closest solution that seems to provide an optimum solution is chosen.
- Greedy algorithms do not always yield optimal solutions, but for many problems they do

Elements of the greedy strategy

- A greedy algorithm obtains an optimal solution to a problem by making a sequence of choices. At each decision point, the algorithm makes choice that seems best at the moment. This heuristic strategy does not always produce an optimal solution, but in some problems, it does.

- How can we tell whether a greedy algorithm will solve a particular optimization problems
No way works all the time, but the greedy-choice property and optimal substructure are the two key ingredients. If we can demonstrate that the problem has these properties, then we are well on the way to developing a greedy algorithm for it.
- The greedy algorithms works if the algorithms has following two properties:
 - **Greedy-choice property:**
 - A globally optimal solution can be reached at by creating locally optimal solution. In other words, an optimal solution can be obtained by performing greedy choices.
 - We can assemble a globally optimal solution by making locally optimal (greedy) choices. In other words, when we are considering which choice to make, we make the choice that looks best in the current problem, without considering results from sub-problems.
 - **Optimal substructure**
 - A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to sub-problems. In other words, optimal solution contains optimal sub-solutions.
 - A problem is said to have optimal substructure if an optimal solution can be constructed from optimal solutions of its sub-problems.

Knapsack Problem

- Given a set of items, each with a weight and a value, determine a subset of items to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.
- We have given a knapsack of capacity W (weight). There are several item $I_1, I_2, I_3, I_4, \dots, I_n$ with different values $V_1, V_2, V_3, V_4, \dots, V_n$ respectively. Each item I_j has weight W_j . Now we want place certain items in the knapsack so that total capacity of knapsack is not exceeded and the total value of items we take must be maximum within the given limit W (i.e. capacity of knapsack)
- If each item taken into knapsack is either whole amount or left it, then such type of problem is called **0-1 knapsack problem**.

- If the fraction of item is allowed to take into knapsack to fulfill the remaining capacity then this type of problem is called **fractional knapsack problem**.
- For the **fractional knapsack problem**, greedy algorithm is used to maximize the value within the limited capacity of knapsack whereas the **0-1 knapsack problem** can be solved using dynamic programming approach.

Algorithm to solve Fractional Knapsack Problem

Given : n items each with different values V_i and of different weights W_i and the knapsack of capacity W . The steps to solve fractional knapsack problem to maximize the value within knapsack capacity can be outlined as:

1. Compute the per unit value of each item i.e. $\frac{V_i}{W_i}$
2. Sort the items based on $\frac{V_i}{W_i}$ in descending order
3. While the knapsack is not full and item list is not empty:
 - If space in knapsack is greater or equal to the weight of item taken then
 - Put the whole item into the knapsack
 - Otherwise,
 - Put the fraction of item that best fit to remaining capacity of knapsack

Analysis of algorithm

- The first step takes n steps to compute the per unit cost. So, time complexity for this step is $O(n)$
- The second step is the sorting process which requires $O(n \lg n)$ by using any efficient sorting technique
- The third step is to choose items from the list and putting into knapsack and this requires at most n steps. So, time complexity for this step is $O(n)$
- The total time complexity = $O(n) + O(n \lg n) + O(n) = O(n \lg n)$

Example:

A thief has just entered into a room and seen different items. Each item has different value and weight. The thief has a knapsack that can carry 20 kg. The items have following values and weights

Piles	1	2	3	4	5	6	7
Weights (Kg)	12	2	3	4	1	1	10
Values(\$)	4	2	2	1	2	3	50

Find which items the thief takes completely and in fraction to maximize the total value?

- Computing the per unit value of each items

Items	1	2	3	4	5	6	7
Weights (Kg) (W_i)	12	2	3	4	1	1	10
Values(\$) (V_i)	4	2	2	1	2	3	50
Value per Kg (V_i/W_i)	1/3	1	2/3	1/4	2	3	5

- Sorting item w.r.t the per unit value of each items

Items	7	6	5	2	3	1	4
Weights (mg) (W_i)	10	1	1	2	3	12	4
Values(\$) (V_i)	50	3	2	2	2	4	1

Capacity of Knapsack, $W = 20$ kg

Here, remaining capacity of knapsack (RC) = $W = 20$ kg

- Choose item 7 with weight = 10 kg which is less than RC
So, put item 7 into the knapsack
 $RC = RC - 10 = 20 - 10 = 10$ kg
- Choose item 6 with weight = 1 kg which is less than RC
So, put item 6 into the knapsack
 $RC = RC - 1 = 10 - 1 = 9$ kg
- Choose item 5 with weight = 1 kg which is less than RC
So, put item 5 into the knapsack
 $RC = RC - 1 = 9 - 1 = 8$ kg

- Choose item 2 with weight = 2 kg which is less than RC
So, put item 2 into the knapsack
 $RC = RC - 2 = 8 - 2 = 6 \text{ kg}$
- Choose item 3 with weight = 3 kg which is less than RC
So, put item 3 into the knapsack
 $RC = RC - 3 = 6 - 3 = 3 \text{ kg}$
- Choose item 1 with weight = 12 kg which is greater than RC
So, take fraction of item 1 (i.e. take 3 kg as remaining capacity is 3kg) and put into the knapsack
 $RC = RC - 3 = 0$
Now the knapsack is full.
So, items in knapsack are = 7,6,5,3,2,1
Total value = $50 + 3 + 2 + 2 + 2 + 3 \cdot (1/3)$ [$\because 12 \text{ kg} = 4, 1 \text{ kg} = 4/12 = 1/3$]
 $= 60$

Job Sequencing with Deadline

— We are given a set of n jobs. Each job has deadline $d_i \geq 0$ and profit $p_i \geq 0$. For any job i profit is earned iff job is completed by deadline. To complete a job one has to process a job for one unit of time. Our aim is to find feasible subset of jobs such that profit is maximum.

Points to considered:

- In this problem we have n jobs $J_1, J_2, J_3, \dots, J_n$. Each job has associated deadline $d_1, d_2, d_3, \dots, d_n$ and profit $p_1, p_2, p_3, \dots, p_n$
- Profit will only be awarded or earned if the job is completed on or before the deadline
- We assume each job takes unit time to complete.
- The objective is to earn maximum profit when only one job can be scheduled or processed at any given time.

Example

Jobs	J1	J2	J3	J4
Profits	100	10	15	27
Deadlines	2	1	2	1

Here, maximum deadline = 2. So, total slot available = 2.

Number of jobs = 4

We can complete only 2 jobs out of 4 jobs within their deadline.

Possible combination of feasible solution for job sequencing:

S.N.	Feasible Solution	Processing Sequence	Total profit
1	(J1, J2)	J2, J1	110
2	(J1, J3)	J1, J3 or J3, J1	115
3	(J1, J4)	J4, J1	127
4	(J2, J3)	J2, J3	25
5	(J3, J4)	J4, J3	42
6	(J1)	J1	100
7	(J2)	J2	10
8	(J3)	J3	5
9	(J4)	J4	27

– The maximum profit is 127 with job sequence J4, J1

- Complexity: $O(n!)$ since all possibilities should be tried

Greedy approach for job sequencing problem

- In the job sequencing with deadline, we can use the greedy strategy such that at each time the job with maximum profit within its deadline is taken as the next job to be added into the partial feasible solution
- In the above example, the job J4 is processed first since its deadline is 1 and J1 is processed after J4 since its deadline is 2. If we process J1 first then we cannot process J4 after J1 because its deadline is already met.
- To choose the job to be placed in the feasible list we should sort the jobs with respect to their profits.
- The greedy strategy process proceeds by sorting all jobs based on their profits and each job is selected for sequencing from the sorted list to get maximum profit.
- The greedy algorithm for job sequencing with deadline can be outlined as follows:

GreedyJobSequencing(n) {

- Sort the jobs with respect to their profit in descending order
- Feasible solution set $J = \{J_1\}$ // first job from the sorted list
- For $i = 2$ to n
 - If all jobs in $J \cup \{J_i\}$ can be completed within deadline then
 - $J = J \cup \{J_i\}$ // add next feasible solution into the partial solution
 - Assign job J_i to the time slot $[r-1, r]$ where r is max integer representing deadline for J_i (i.e. available time slot within deadline). If not such slot then do not schedule J_i

Complexity Analysis

- Sorting the job with respect to profit requires $O(n \lg n)$ time
- Choosing first job to the feasible solution set and assigning to a time slot is done in constant time i.e. $O(1)$
- Choosing the job from list and assigning into solution set depends on the number of jobs in the solution list.
 - Let k be the number of jobs in the solution sequence
 - So, complexity is $O(k*n)$ and in worst case $k = n$ so complexity is $O(n^2)$
- Hence, total time complexity for this algorithm is $= O(n \lg n) + O(1) + O(n^2)$
 $= O(n^2)$

Example:

Let us consider a set of given jobs as shown in the following table. We have to find a sequence of jobs, which will be completed within their deadlines and will give maximum profit. Each job is associated with a deadline and profit.

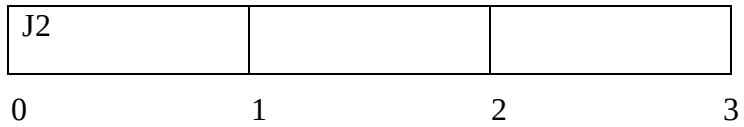
Jobs	J₁	J₂	J₃	J₄	J₅
Deadlines	2	1	3	2	1
Profits	60	100	20	40	20

To solve this problem, the given jobs are sorted according to their profit in a descending order. Hence, after sorting, the jobs are ordered as shown in the following table.

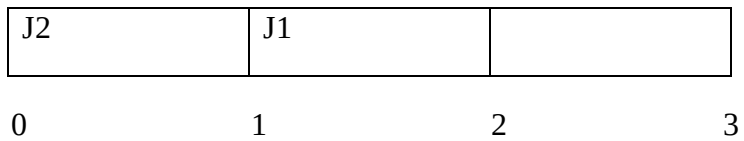
Jobs	J₂	J₁	J₄	J₃	J₅
Deadlines	1	2	2	3	1
Profits	100	60	40	20	20

Here maximum deadline = 3 so, 3 time slots

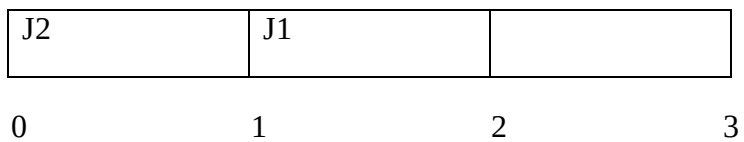
- From this set of jobs, first we select J2, as it can be completed within its deadline and contributes maximum profit.



- Next, J1 is selected as it can be completed within its deadline



- In the next step, J4 cannot be selected as its deadline is over, hence J3 is selected as it executes within its deadline.



So, Selected jobs = J1, J2, J3

Sequence = J2 , J1, J3

Profit = $100 + 60 + 20 = 180$

Huffman Coding

- Huffman coding is a lossless data compression technique.
- Huffman codes are used to compress data by representing each alphabet by unique binary codes in an optimal way.
- Suppose we have a 100,000-character data file that we wish to store compactly. We observe that the characters in the file occur with the frequencies given by following table. That is, only 6 different characters appear, and the character **a** occurs 45,000 times.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5

- We have many options for how to represent such a file of information. Here, we consider the problem of designing a *binary character code* (or *code* for short) in which each character is represented by a unique binary string, which we call a *codeword*. If we use a **fixed-length code**, we need 3 bits to represent 6 characters: a = 000, b = 001, . . . , f = 101. This method requires 300,000 bits to code the entire file.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

- A **variable-length code** can do considerably better than a **fixed-length code**, by giving *frequent characters short codewords* and *infrequent characters long codewords*.
- The table below shows such a code; here the 1-bit string 0 represents **a**, and the 4-bit string 1100 represents **f**.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

This code requires

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1,000 = 224,000 \text{ bits}$$

to represent the file, a savings of approximately 25%.

In fact, this is an optimal character code for this file.

Prefix codes

- Prefix code is a code in which no codeword is also a prefix of some other codeword.
- A prefix code is a binary code assigned to any character in the file for encoding such that no any code is the prefix of another code
- For example: a = 0, b = 10, c = 110, d = 1110, e = 11111, f = 111101 , g = 111100
- A prefix code can always achieve the optimal data compression among any character code.
- Huffman algorithm always generates the prefix code for encoding the file into binary code so that there is no conflict during the decoding process.
- Since no codeword is a prefix of any other, the codeword that begins an encoded file is unambiguous.

Note: Perhaps “prefix-free codes” would be a better name, but the term “prefix codes” is standard in the literature.

Huffman tree

- Huffman tree or Huffman coding tree defines as a binary tree in which each leaf of the tree corresponds to a letter.

Huffman Coding Algorithm

- Huffman invented a **greedy algorithm** that constructs an optimal prefix code called a Huffman code.
- A tree corresponding to optimal code is constructed from bottom up manner starting from $|C|$ nodes as leaves where C is a set of distinct characters into file.
- Uses priority queue data structure to keep nodes ordered by frequency where the priority queue used by the algorithm is binary heap. The algorithm uses a min-priority queue Q, keyed on the frequency attribute, to identify the two least-frequent objects to merge together.

Algorithm

```

Huffman(C) { // C is set of distinct characters
    n = |C|;
    Q = C; // Q is priority queue as binary heap
    for( i= 1, i<n ; i++)
    {
        z = AllocateNode( );
        x = ExtractMin(Q) ; // Extract min node from heap
        y = ExtractMin(Q);
        left(z) = x;
        right(z) = y;
        f(z) = f(x) + f(y) ; // f(z) is frequency of z
        Insert(z,Q); // insert z to Q
    }
    return ExtractMin(Q); // return the root of the tree
}

```

Here, $n = |C|$ is a initial queue size, $(n-1)$ merge steps are required to build the tree. The final tree represents the optimal prefix code. The codeword for a letter is the sequence of edge labels on the path from the root to the letter (labels are set as 0 for left and 1 for right or vice-versa) .

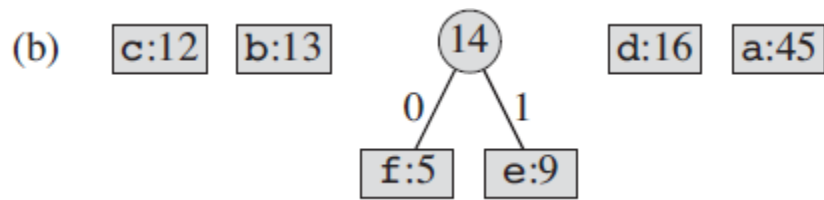
Example

Characters (C)	a	b	c	d	e	f
Frequencies (f(C))	45	13	12	16	9	5

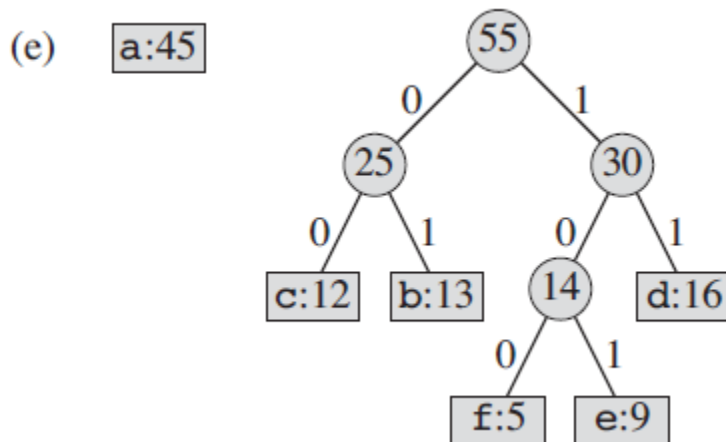
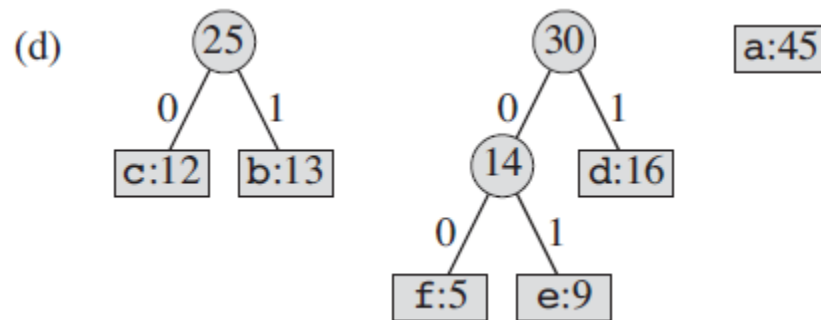
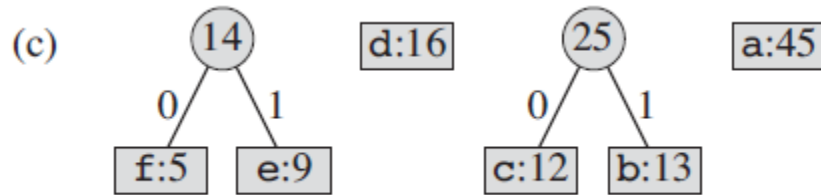
- The initial set of $n = 6$ nodes, one for each letter. (Initial priority queue)

(a) f:5 e:9 c:12 b:13 d:16 a:45

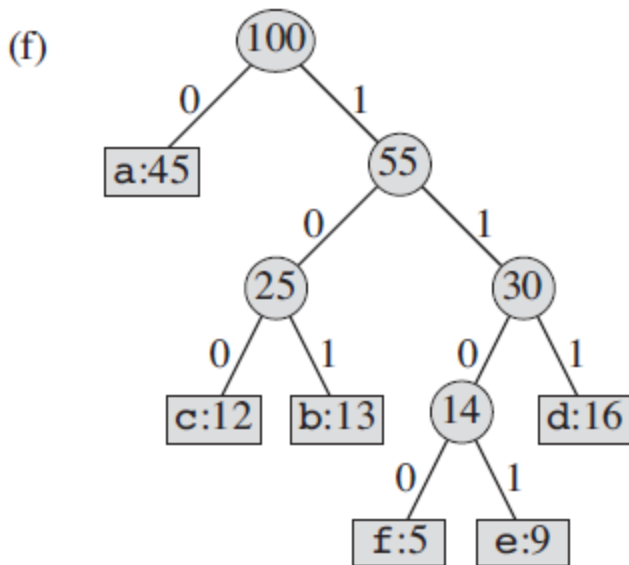
- Extracting the two nodes x and y of lowest frequency from the queue, replacing them in the queue with a new node z representing their merger. The frequency of z is computed as the sum of the frequencies of x and y . Label 0 to left branch and 1 to right branch.



Repeating the merging process



Now the final tree is:



Now the prefix codes (combining labels from root to leaf) with bit length can be listed in following table

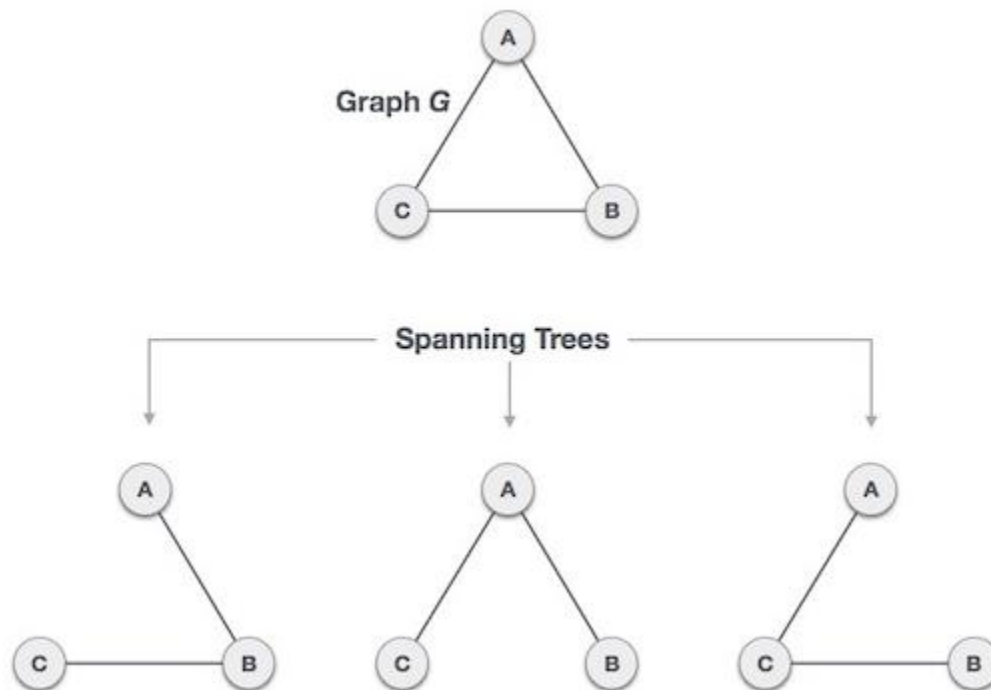
Character	Prefix code	Bit length	Frequency	Total
a	0	1	45	$45 \times 1 = 45$
b	101	3	13	$3 \times 13 = 39$
c	100	3	12	$3 \times 12 = 36$
d	111	3	16	$3 \times 16 = 48$
e	1101	4	9	$4 \times 9 = 36$
f	1100	4	5	$4 \times 5 = 20$
Total Numbers of bits required				224

Analysis

- For a set C of n characters, we can initialize Q in line 2 in $O(n)$ time using the `BuildMinHeap()` procedure
- The for loop executes exactly $(n - 1)$ times, and since each heap operation requires $O(\lg n)$ time, the loop contributes $O(n \lg n)$ to the running time.
- Thus, the total running time of *Huffman()* algorithm on a set of n characters is **$O(n \lg n)$**

Spanning Trees

- The Spanning tree of a graph is a subgraph that contains all the vertices connected by minimum number of edges and is also a tree.
- A spanning tree does not have any cycle
- A graph can have more than one spanning tree.
- A spanning tree has $n - 1$ edges. Where n is the number of vertices
- From a complete graph, by removing maximum $(e - n + 1)$ edges, we can construct a spanning tree. Here e is number of edges and n is number of vertices in a graph.
- A complete undirected graph can have n^{n-2} number of spanning trees where n is the number of vertices in the graph



- All possible spanning trees of a graph G , have the same number of edges and vertices.
- Removing one edge from the spanning tree will make the graph disconnected
- Adding one edge to the spanning tree will create a circuit or loop
- If all the vertices are connected in a graph, then there exists at least one spanning tree.
- Spanning trees are a subset of connected graph and disconnected graphs do not have spanning tree

Minimum Spanning Tree (MST)

- A minimum spanning tree is a spanning tree in which the sum of the weights of its edges is minimal.
- A minimum spanning tree of a graph G is a tree formed from graph edges that connect all the vertices of G with minimum total cost (weights).
- **Minimum spanning tree of a graph G is a tree that includes all the nodes of a graph G and has the minimum cost among all spanning trees of the Graph G .**
- Popular algorithms to form a MST
 - Kruskal's Algorithm
 - Prim's Algorithm

Kruskal's Algorithm for Minimum Spanning Tree

- Kruskal's Algorithm is a famous greedy algorithm.
- Devised by Joseph Kruskal.
- It is used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply Kruskal's algorithm, the given graph must be weighted, connected and undirected.
- In this method, all edges are ordered by weight, and then each edge in this ordered sequence is checked to see whether it can be considered part of the tree under construction. It is added to the tree if no cycle arises after its inclusion.

The main steps in Kruskal's algorithm are as follows:

- Sort all the edges from low weight to high
- Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
- Keep adding edges until we reach all vertices.

Algorithm

MST-KRUSKAL(G, w)

```

1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 

```

Here, MAKE-SET, FIND-SET and UNION are the operation of Disjoint Set data structure

Disjoint Set Data Structure

A **disjoint-set** data structure, also called a **union–find** data structure or **merge–find set**, is a data structure that stores a collection of disjoint (non-overlapping) sets. Equivalently, it stores a partition of a set into disjoint subsets. It provides operations for adding new sets, merging sets (replacing them by their union), and finding a representative member of a set. The last operation allows to find out efficiently if any two elements are in the same or different sets.

In disjoint set data structure, following three operations are required:

- a) **MAKE-SET** (u) : Makes a new singleton set with an element u

[Singleton set : the set with only one element]

Example:

MAKE-SET (a)

$S1 = \{a\}$

MAKE-SET (b)

$S2 = \{b\}$

- b) **UNION** (u, v) : Merges two sets u and v

Example:

UNION ($S1, S2$)

If $S1 = \{a\}$ and $S2 = \{b\}$

$S3 = \{a,b\}$ is returned

c) FIND-SET (v) : Returns the representative of the set v.

- In disjoint sets the elements are always arranged in Lexicographically order(alphabetic order) or in increasing order if the elements are numeric values.
- Therefore, the set representative will always be the first element of the set.

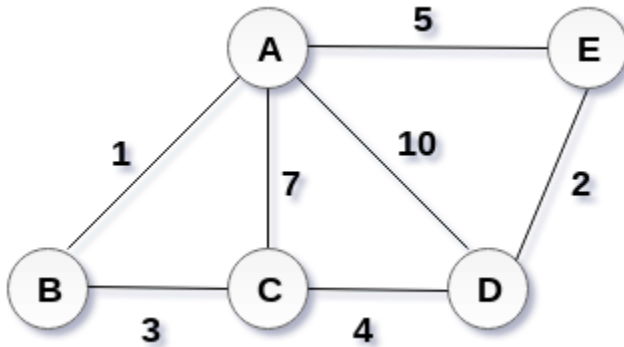
Example:

$S3 = \{a,b,c\} \Rightarrow$ representative of set S3 is a

$S4 = \{5,6,9\} \Rightarrow$ representative of set S3 is 5

FIND-SET (S3) $\Rightarrow$ a

Example: Constructing a MST from given graph using Kruskal's Algorithm



The graph contains 5 vertices and 7 edges. So, the minimum spanning tree formed will be having $(5 - 1) = 4$ edges.

The weights of the edges are:

Edges	AE	AD	AC	AB	BC	CD	DE
Weights	5	10	7	1	3	4	2

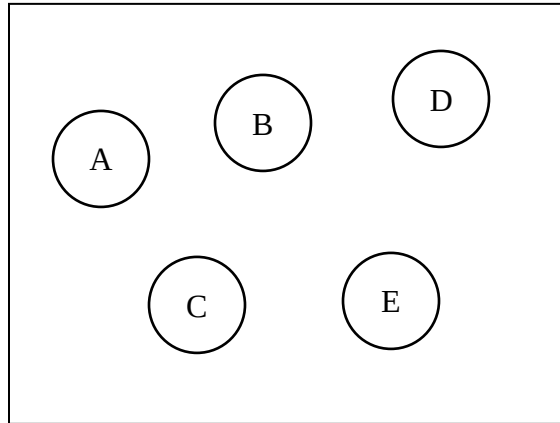
Edges in ascending order of their weight are:

Edges	AB	DE	BC	CD	AE	AC	AD
Weights	1	2	3	4	5	7	10

So, Set E = {AB, DE, BC, CD, AE, AC, AD}

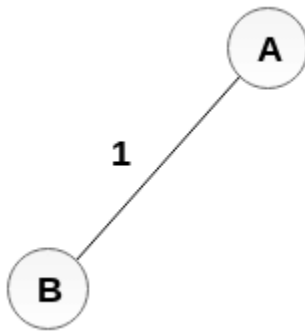
Initially the forest of single node tree is constructed a

for each vertex $v \in G.V$
 MAKE-SET(v)

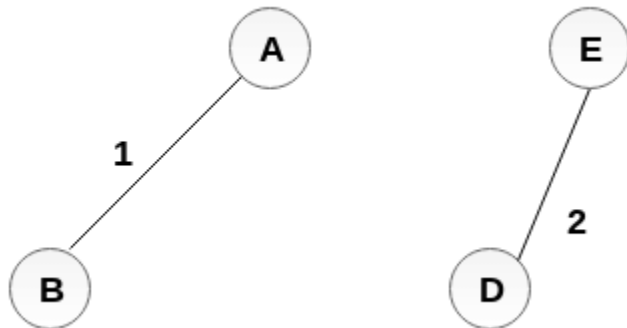


Start constructing the tree;

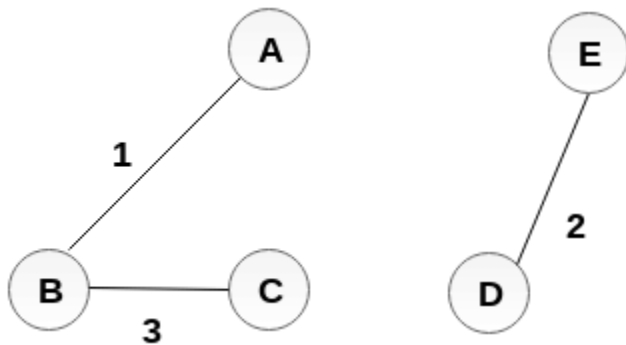
Adding AB to MST because adding this does not form a cycle;



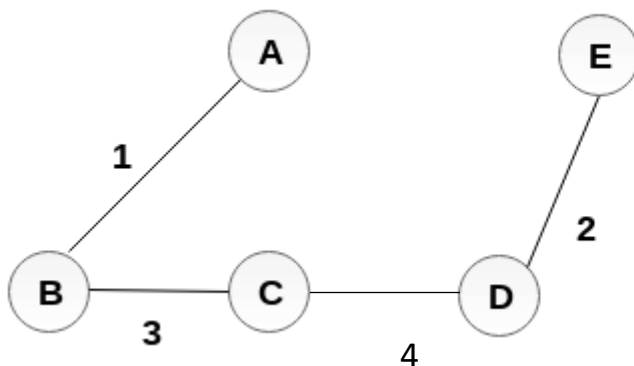
Now adding DE to MST; because adding this does not form a cycle



Now Adding BC to MST



Next, CD is added to MST



Now, number of edges in MST has become 4 (i.e. $|tree| = |v|-1$). Here, all the vertices are included in MST So we stop here.

Here the sum of all edge weights in this MST = $1+2+3+4 = 10$

Analysis

The running time of Kruskal's algorithm for a graph $G(V, E)$ depends on how we implement the disjoint-set data structure.

- Initializing the set A in line 1 takes $O(1)$ time
- Step 2-3 takes $O(V)$ time
- The time to sort the edges in line 4 is $O(E \lg E)$.

- The for loop of lines 5–8 performs $O(E)$ FIND-SET and UNION operations on the disjoint-set forest. The time complexity of these steps is $O(E \lg V) = O(E \lg E)$ (refer book for detail)
- So total time complexity = $O(1) + O(V) + O(E \lg E) + O(E \lg E)$
 $O(E \lg E)$

✎ We can also represent the complexity in term of V which can be shown as follow.

We know $|E| < |V|^2$

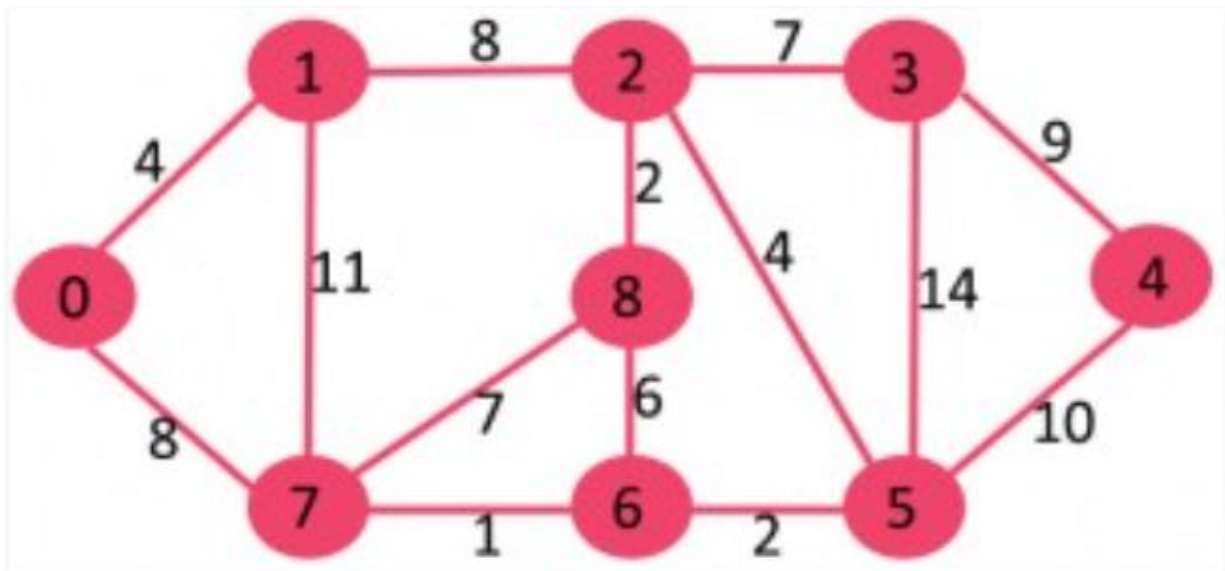
Taking log on both sides

$$\lg E < \lg V^2$$

$$\lg E < 2\lg V = O(\lg V)$$

So, Time complexity = $O(E \lg V)$

Exercise: What is the cost of MST of given graph? Calculate by using Kruskal's Algorithm.



Prim's Algorithm for Minimum Spanning Tree

- Like Kruskal's algorithm, Prim's algorithm is also a Greedy algorithm.
- It starts with an empty spanning tree.
- In contrast to Kruskal's algorithm that starts with graph edges, Prim's algorithm starts with a vertex. We start with one vertex and keep on adding edges with the least weight till all the vertices are covered.
- The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included.
- At every step, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges.
- Prim's algorithm works by maintaining two sets R and $(V-R)$ where V is the set of vertices and R is a sub set of V .
- It starts with arbitrary vertex and checks all the edges from that vertex to other vertices and minimum cost edge is taken at next item to be added in R .
- Initially R is empty and starts with taking a node in R .

Algorithm

```

PrimsMST(G){
    T =  $\phi$ ;
    R = {vi}; // vi is an arbitrary node from set of all vertices, V
    while( R!=V){
        choose edge e = (u,v) where u is from R , v is from (V-R) and e is of minimum weight;
        add v to R i.e. R = R  $\cup$  {v} and e to T i.e T = T  $\cup$  e
    }
    return T;
}

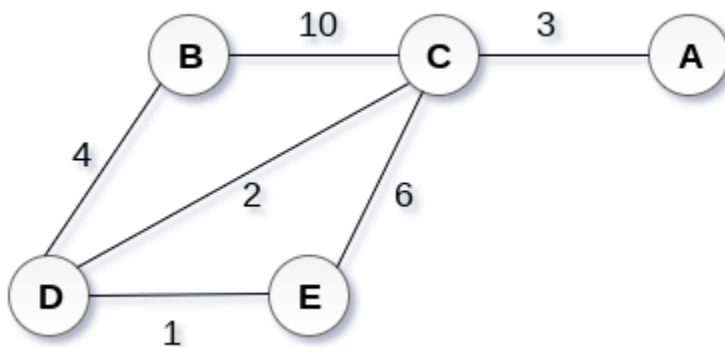
```

Note: refer book for more detail version of pseudocode which will be helpful for coding

The main steps in Prim's algorithm are as follows:

- Choose a random vertex as starting vertex and initialize a minimum spanning tree.
- Find the edges that connect to other vertices. Find the edge with minimum weight and add it to the spanning tree.
- Repeat step 2 until the spanning tree is obtained.

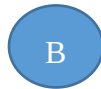
Example:



Set of all vertices $V = \{A, B, C, D, E\}$

Step 1: Let's choose B as initial vertex

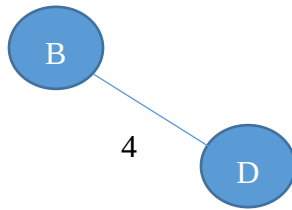
So, $R = \{B\}$



Step 2:

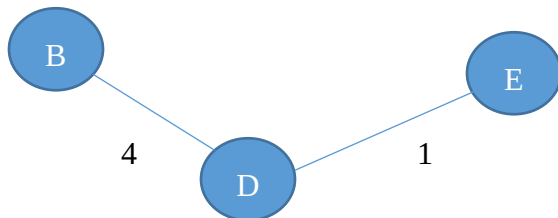
R	V-R	Possible edges with weights (From R to V-R) that are not included in MST	Edge with minimum weight
{ B }	{ A, C, D, E }	BC = 10 ; BD = 4	BD

Minimum cost edge BD is added to MST and vertex D is added to R . So, now $R = \{B, D\}$ and $V-R = \{ A, C, E \}$

**Step 3:**

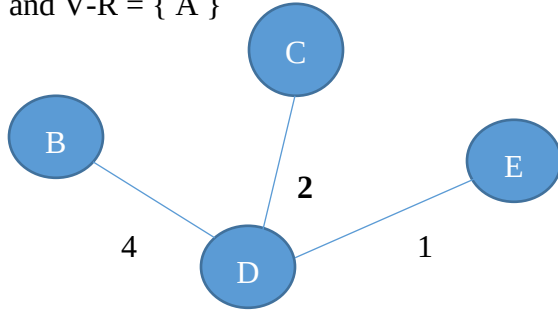
R	V-R	Possible edges with weights (From R to V-R) that are not included in MST	Edge with minimum weight
{ B, D }	{ A, C, E }	BC = 10 ; CD = 2; DE = 1	DE

Minimum cost edge DE is added to MST and vertex E is added to R . So, now $R = \{B, D, E\}$ and $V-R = \{ A, C \}$

**Step 4:**

R	V-R	Possible edges with weights (From R to V-R) that are not included in MST	Edge with minimum weight
{ B, D, E }	{ A, C }	BC = 10 ; CD = 2; CE = 6	CD

Minimum cost edge CD is added to MST and vertex C is added to R . So, now $R = \{B, D, E, C\}$ and $V-R = \{A\}$



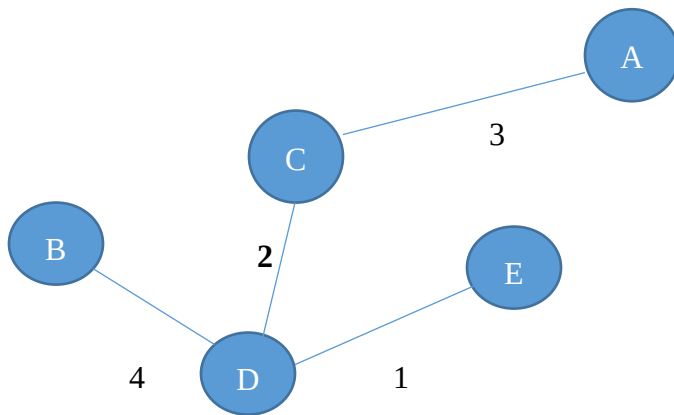
Step 5:

R	V-R	Possible edges with weights (From R to V-R) that are not included in MST	Edge with minimum weight
{ B, D, E, C }	{ A }	BC = 10 ; CE = 6; CA= 3	CA

Minimum cost edge CA is added to MST and vertex A is added to R . So, now

$R = \{B, D, E, C, A\}$ and $V-R = \{ \}$

Here $V = R$ so, we stop.



The cost of MST = $4 + 1 + 2 + 3 = 10$

Analysis

In Prim's algorithm, the edge list connecting the set R and $(V-R)$ can be maintained in a priority queue using suitable data structure.

- In the algorithm, the loop for adding minimum edge to R executes in $O(V)$
- The edge with minimum weight can be extracted from the priority queue in $O(E)$
- So, Total time complexity using ordinary priority queue data structure takes $O(EV)$
- But using the priority queue as a min heap data structure, the extract minimum operation can be reduced to $O(\lg V)$
- So, if binary min heap data structure is used, then the total time complexity will be $O(E \lg V)$

Shortest path problem

- The shortest path problem is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.
- **Single-source shortest- paths problem:** Find the shortest paths between a given vertex and all other vertices in the graph. Given a graph $G(V, E)$, we want to find a shortest path from a given source vertex $s \in V$ to each vertex $v \in V$.
- **Single-destination shortest-paths problem:** Find a shortest path to a given destination vertex t from each vertex v . By reversing the direction of each edge in the graph, we can reduce this problem to a single-source problem.
- **Single-pair shortest-path problem:** Find a shortest path from u to v for given vertices u and v . If we solve the single-source problem with source vertex u , we solve this problem also.
- **All-pairs shortest-paths problem:** Find a shortest path from u to v for every pair of vertices u and v

Dijkstra's Shortest- path Algorithm

- One of the most popular algorithm to solve single source shortest paths problem which is based on greedy approach.

- Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G(V, E)$ for the case in which all edge weights are nonnegative
- Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined. The algorithm repeatedly selects the vertex $u \in (V - S)$ with the minimum shortest-path estimate, adds u to S , and relaxes all edges leaving u .
 - Set S is the set of vertices to which the shortest path from the source vertex is already known
 - At each step, the algorithm adds a vertex u from $(V-S)$ to S such that the distance from source vertex to u is minimum among all vertices in $(V-S)$

Algorithm:

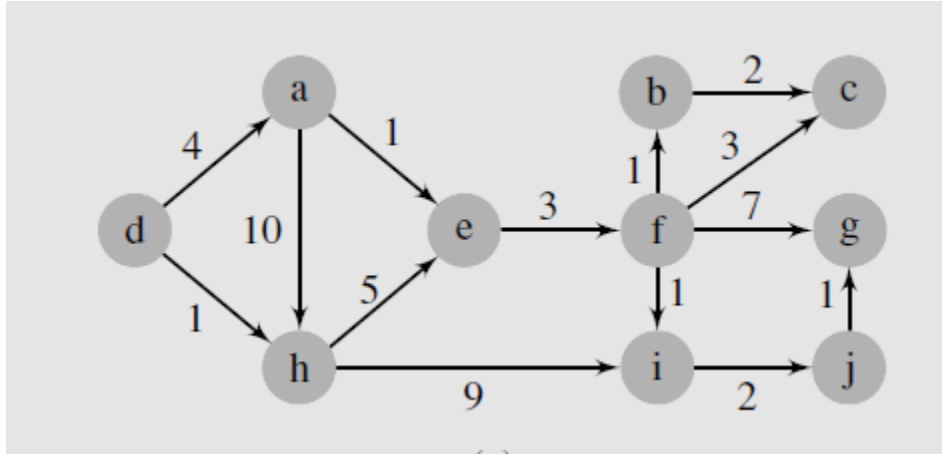
```

Dijkstra( $G, w, s$ ) //  $G$  is graph ,  $w$  is set of weights ,  $s$  is source vertex
{
    for each vertex  $v \in V$ 
         $d(v) = \infty$  ; // current distance of vertex  $v$  from source is set to  $\infty$ 
     $d(s) = 0$  ; // current distance of vertex  $s$  (source) is set to 0
     $S = \Phi$ ; // initially  $S$  is empty
     $Q = .V$ ; // min-priority queue  $Q$  of vertices keyed by their  $d$  (distance) values
    while( $Q \neq \Phi$ )
    {
         $u = \text{ExtractMin}(Q)$ ; // take minimum from  $Q$  and delete
         $S = S \cup \{u\}$ ; // add  $u$  to  $S$ 
        for each vertex  $v$  adjacent to  $u$ 
            // relaxation
            if  $d(v) > d(u) + w(u, v)$  //  $w(u, v)$  is weight of edge  $(u, v)$ 
                 $d(v) = d(u) + w(u, v)$ ;
    }
    return  $d(v)$  where  $v \in V$ ;
}

```

Example:

Trace Dijkstra's single source shortest path algorithm for following graph where vertex d is the source vertex.



The weight adjacency matrix of given graph

	a	b	c	d	e	f	g	h	i	j
a	0	∞	∞	∞	1	∞	∞	10	∞	∞
b	∞	0	2	∞	∞	∞	∞	∞	∞	∞
c	∞	∞	0	∞	∞	∞	∞	∞	∞	∞
d	4	∞	∞	0	∞	∞	∞	1	∞	∞
e	∞	∞	∞	∞	0	3	∞	∞	∞	∞
f	∞	1	3	∞	∞	0	7	∞	1	∞
g	∞	∞	∞	∞	∞	∞	0	∞	∞	∞
h	∞	∞	∞	∞	5	∞	∞	0	9	∞
i	∞	∞	∞	∞	∞	∞	∞	∞	0	2
j	∞	∞	∞	∞	∞	∞	1	∞	∞	0

Initially current distance of vertices $d(v)$ from source is set to ∞ and the current distance of source i.e. $d(d)$ is set to 0. Similarly The set $S = \{\}$

The source d is first selected since it has the minimum current distance.

$S = \{d\}$

Adjacent vertex of d = { a , h }

Relaxation rule :

$$d(v) > d(u) + w(u,v) \text{ then } d(v) = d(u) + w(u,v)$$

When vertex a is considered

$$d(a) > d(d) + w(d,a)$$

$$\infty > 0 + 4 \text{ is true so } d(a) = 0 + 4 = 4$$

When, vertex h is considered

$$d(h) > d(d) + w(d,h)$$

$$\infty > 0 + 1 \text{ is true so } d(h) = 0 + 1 = 1$$

Here, $\min(d(a), d(h)) = \min(4, 1) = 1$ So, h is extracted and added to S.

The adjacent vertex with minimum d value is h, So, vertex h is selected and added to set S.

$$S = \{d, h\}$$

Adjacent vertex of h = { e , i }

Similar operations are carried out in other iterations until all the vertices are examined. The updated current distance values of each vertices and the contents of set S in each iteration are expressed in following table:

Iteration	Selected Vertetex	a	b	c	d	e	f	g	h	i	j	Set S
initial	—	∞	∞	∞	0	∞	∞	∞	∞	∞	∞	{ }
1	d	4	∞	∞		∞	∞	∞	1	∞	∞	{d}
2	h	4	∞	∞		6	∞	∞		10	∞	{d,h}
3	a		∞	∞		5	∞	∞		10	∞	{d,h,a}
4	e		∞	∞			8	∞		10	∞	{d,h,a,e}
5	f		9	11				15		9	∞	{d,h,a,e,f}
6	b			11				15		9	∞	{d,h,a,e,f,b}
7	i			11				15			11	{d,h,a,e,f,b,i}
8	c							15			11	{d,h,a,e,f,b,i,c}
9	j							12				{d,h,a,e,f,b,i,c,j}
10	g											{d,h,a,e,f,b,i,c,j,g}

Hence the single source shortest path cost for source d can be listed as

(Source, Destination)	Cost
(d ,d)	0
(d ,h)	1
(d ,a)	4
(d ,e)	5
(d ,f)	8
(d ,b)	9
(d ,i)	9
(d ,c)	11
(d ,j)	11
(d ,g)	12

Analysis

- In the above algorithm, the first for loop block takes $O(V)$ time.
- Initialization of Set S and d(s) takes $O(1)$
- Initialization of priority queue Q takes $O(V)$ time.
- The while loop executes for $O(V)$, where for each execution the block inside the loop takes $O(V)$ times .
- Hence the total running time is $O(V^2)$.